Worksheet — 2.1 Elements of Computational Thinking

Name: __ **Date:** ____

OCR A Level Computer Science (H446) — Component 02 — Spec ref 2.1.1–2.1.5

Time: ~60 min.

> Reminder: this topic is examined by **application to a scenario**, not by reciting definitions. Always name the scenario's *actual* inputs, conditions and sub-procedures.

The scenario — *BookIt* library room-booking system

Throughout this worksheet you will work on one scenario: a small program called **BookIt** that lets students book a study room in a library. A student picks a room and a time slot, and the system either confirms the booking or rejects it. It shows the rooms that are free, takes the student's ID and chosen slot, checks availability, then stores the booking and prints a confirmation.

Keep this scenario in mind for every task below.

Section A — Skills practice

Task 1 — Key-term match (the five thinking skills)

Draw a line (or write the matching letter) to connect each thinking skill to its definition.

Thinking skill		Definition
1. Thinking abstractly		A. Identifying decision points where the flow of control branches on a condition.
2. Thinking ahead		B. Spotting parts of a problem that can run at the same time.
3. Thinking procedurally		C. Removing/hiding unnecessary detail so only relevant information remains.
4. Thinking logically		D. Breaking a problem into an ordered sequence of steps and sub-procedures.
5. Thinking concurrently		E. Planning before coding: inputs/outputs, preconditions, caching, reusable components.

Answers: 1 ___ 2 ___ 3 ___ 4 ___ 5 ___

Task 2 — Thinking ahead: inputs, outputs and preconditions (BookIt)

(a) List **three inputs** the BookIt system needs from the user.

1.

2.

3.

(b) List **two outputs** the system produces.

1.

2.

(c) State **one precondition** that must be true *before* a booking can be confirmed.

> Tip: "Validate the student ID" is a **process**, not an input or output — don't list it in (a) or (b).

Task 3 — Abstraction: keep or remove? (BookIt)

You are modelling each *room* in BookIt. For each piece of detail below, tick whether you would **KEEP** it in the model or **REMOVE** it, and write a one-line reason for the first three.

Detail about a room	KEEP	REMOVE
Room number / ID	☐	☐
Maximum capacity (seats)	☐	☐
Colour of the carpet	☐	☐
Which time slots are already booked	☐	☐
The make of chairs in the room	☐	☐

Reasons:

KEEP example — _____

REMOVE example — _____

Why is the removed detail irrelevant **to this problem** specifically? _____

Task 4 — Decomposition: break it into sub-problems (BookIt)

Decompose the "make a booking" task into **four to six** sub-problems / sub-procedures. Then number them in the order they must run.

Order	Sub-procedure
____	_____
____	_____
____	_____
____	_____
____	_____
____	_____

Which sub-procedure **depends on the output** of an earlier one? State which depends on which.

Task 5 — Thinking logically: decision points (BookIt)

Identify **two decision points** in BookIt. For each, write the **exact Boolean condition** and state what happens in **both** branches.

Decision point 1

Condition (Boolean): `IF _____`

If TRUE: _____

If FALSE: _____

Decision point 2

Condition (Boolean): `IF _____`

If TRUE: _____

If FALSE: _____

Task 6 — Thinking concurrently: what can run at once? (BookIt)

(a) Name **two processes** in BookIt that could run **concurrently** because they have **no data dependency** between them.

1.
2.

(b) Name **two steps that must stay sequential** (one depends on the other) and say why.

(c) State **one benefit** and **one trade-off** of running parts of BookIt concurrently.

Benefit: _____

Trade-off: _____

Task 7 — Abstraction vs decomposition: which is which?

For each statement, write **A** (abstraction) or **D** (decomposition).

1. Ignoring the carpet colour when modelling a room. ____
2. Splitting "make a booking" into check-availability, store-booking and confirm. ____
3. Representing the whole library as a list of rooms with free/busy flags. ____
4. Breaking the program into separate display, validate and save procedures. ____
5. Treating "savings" and "current" accounts as one general `Account` model. ____
6. Dividing the timetable problem into "morning slots" and "afternoon slots" handlers. ____

Task 8 — All five thinking skills in ONE scenario

> **New scenario — *SnackTrack*.** A school canteen installs a self-service kiosk. A student taps their ID card, the kiosk shows today's menu with a photo and price for each item (it does **not** show the recipe, supplier or kitchen rota), the student picks items, the kiosk checks the student has enough credit **and** that the item is in stock, takes payment from the account, updates the stock level, and prints a receipt — all while a second thread refreshes the "sold out" labels on the other kiosks.

In the exam you may be asked to identify **how each element of computational thinking is used** in a scenario like this. Complete the table — every example must come **from SnackTrack**, not from a definition.

Thinking skill	One concrete example from SnackTrack
Thinking **abstractly**	
Thinking **ahead**	
Thinking **procedurally**	
Thinking **logically**	
Thinking **concurrently**	

Self-check: if any row of yours would make sense with the words "SnackTrack" deleted, it is a definition, not an application — rewrite it.

Task 9 — Precondition, caching, or neither?

Thinking ahead includes identifying **preconditions** (what must already be true for a step to work correctly) and deciding what to **cache** (store ready for reuse so it does not have to be recomputed/re-fetched). For each statement about **BookIt** or **SnackTrack**, tick ONE column.

#	Statement	Precondition	Caching	Neither
1	The student's ID must exist in the accounts database before a purchase can be processed.	☐	☐	☐
2	Today's menu (with photos) is downloaded once at 7 am and kept in the kiosk's memory all day.	☐	☐	☐
3	The chosen time slot must still be free at the moment the booking is written to the database.	☐	☐	☐
4	The kiosk prints a receipt after payment succeeds.	☐	☐	☐
5	The "rooms currently free" list is stored in RAM and reused for every student's request instead of re-querying the database.	☐	☐	☐
6	The room list array must be **sorted** before a binary search can be used to find a room.	☐	☐	☐
7	The kiosk screen is a touchscreen.	☐	☐	☐

(b) For **one** caching row you ticked, state what event would make the cached data **stale**.

(c) For **one** precondition row you ticked, state what the program should do if the precondition is **false**.

Challenge box

> **Caching in BookIt.** The "rooms currently free" list is requested by many students at once and is expensive to recompute from the database every time. Explain how **caching** this list would help, and give **one trade-off**, including what could make the cached data go **stale** and how you would handle it.

Section B — Exam practice (30 marks)

Answer **all** questions. The question marked with an asterisk (*) is marked for the quality of your extended response. Answers must be applied to the ParkSmart scenario — generic definitions gain few marks.

> **Scenario — ParkSmart.** ParkSmart is a computerised multi-storey car park. A camera at the entrance reads each car's **number plate**. The system checks whether the car park is full; if not, it assigns the driver a **free bay** and displays the bay number on a screen, then raises the barrier. Sensors in every bay report whether the bay is occupied. When leaving, the driver enters their number plate at a pay station, the system calculates the **fee from the length of stay**, takes card payment, and raises the exit barrier. Digital signs around town display the number of free spaces, updated every 60 seconds.

Q1. Describe what is meant by **thinking ahead** when planning a program such as ParkSmart. **[2]** (AO1)

Q2. Identify **two** inputs to the ParkSmart system. **[2]** (AO2)

Q3. The programmers model each parking bay simply as a bay number and an occupied/free flag, ignoring details such as the bay's paint colour and floor surface. Explain how this is an example of **abstraction** and why it is appropriate for ParkSmart. **[3]** (AO2)

Q4. The "handle a car entering" problem can be **decomposed** into sub-problems. Identify **four** sub-problems, in the order they must be carried out. **[4]** (AO2)

Q5. Identify **one decision point** in ParkSmart. Write its condition as a **Boolean expression** and state the outcome of **each** branch. **[3]** (AO2)

Q6. The town-centre signs showing the number of free spaces are updated from a value **cached** in the system's memory, rather than by counting all the bay sensors every time. Explain **one** benefit and **one** drawback of this use of caching in ParkSmart. **[3]** (AO2)

Q7. ParkSmart must serve the entrance camera, the exit pay stations and the bay sensors **at the same time**. Explain how **thinking concurrently** applies here, giving **one** example of two ParkSmart processes that can safely run concurrently and **one** problem that concurrency could cause in this system. **[4]** (AO2)

Q8* The developers of ParkSmart are deciding between a **single sequential program** that handles one event at a time and a **concurrent design** in which entrance, exit and sensor updates are processed simultaneously. Discuss the benefits and drawbacks of using a concurrent design for ParkSmart, and recommend, with justification, which design the developers should use. **[9]** (AO3)

(Total for Section B: 30 marks)